

Storage class in C

27 June 2026 20:21

A storage class in C specifies how, where and for how long a variable or function exists in memory. It determines four important properties of a variable:

1. **Storage Location:** Where the variable is stored (memory segment).
2. **Scope(Visibility):** Where the variable can be accessed.
3. **Lifetime(Storage duration):** How long the variable remains in memory.
4. **Linkage:** Whether the variable or function can be accessed from other source files.

Why do we need storage Classes?

Consider the following situations:

- Some variables are required only inside a function.
- Some variables must retain their values between function calls.
- Some variable needs to be shared among multiple source files.
- Some variables should be accessible only within a single source file.

Storage classes help the compilers manage these requirements efficiently.

Types of Storage

There are four primary storage classes in C:

1. **auto**
2. **register**
3. **static**
4. **extern**

1. **Automatic Storage Class(auto):** The auto storage class is the **default storage class** for local variables. Whenever a variable is declared inside a function without specifying a storage class, it is automatically an **auto** variable. For example: `int x;` or `auto int x;` Both the declarations are equivalent.

Characteristic:

- Stored in **main memory (RAM)**
- Scope is limited to the block or function.
- Lifetime is only during the execution of the block.
- Initial value is garbage (intermediate).

```
#include<stdio.h>
void f1(){
    int x = 10;
    printf("\nx = %d\n", x);
}
// Once f1() finished 'x' is destroyed.
int main(int argc, char const *argv[])
{
    f1();
}
```

```

    return 0;
}

```

```

1 // Online C compiler to run C program online
2 #include <stdio.h>
3 void f10{
4     auto int x = 10;
5     printf("\nx = %d\n", x);
6 }
7 int main0 {
8     // Write C code here
9     printf("Hello, World!");
10    f10;
11    return 0;
12 }

```

```

Hello, World!
x = 10

```

2. Register Storage Class (register): The register variable request the compiler to store the variable in a CPU register instead of RAM for faster access.

Syntax:

```
register int count;
```

Characteristics:

- Stored in CPU register. (if available)
- Faster access than RAM.
- Scope is local to the block.
- Lifetime is only during block execution.
- Initial value is garbage.
- Cannot use the address operator(&).

```

#include<stdio.h>
int main(int argc, char const *argv[])
{
    register int i;
    printf("\n");
    for(i = 1; i <= 5; i++){
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}

```

Output:

```
1 2 3 4 5
```

Important Note: This is invalid because register variable may not have a memory address accessible to the program.

3. Static Storage Class(static): A static variable is initialized only once and

retain its value throughout the program execution. Unlike automatic variables, it is **not destroyed** when the function ends.

Syntax:

```
static data_type variable;
```

Characteristics:

- Stored in the data segment.
- Scope depends on where it is declared.
- Lifetime is the entire program.
- Default value is zero for integer types Or dependent upon a data type.

```
#include<stdio.h>
void counter(){

    static int count = 0; //execute only once for the first call only.
    count++;
    printf("%d\n",count);
}
int main(int argc, char const *argv[])
{
    counter();
    counter();
    counter();
    counter();
    return 0;
}
```

Output:

```
1
2
3
4
```

The variable is not recreated every time the function executes.

Without static keyword:

```
#include <stdio.h>
void counter()
{
    int count = 0;
    count++;
    printf("%d\n", count);
}
int main(int argc, char const *argv[])
{
    counter();
    counter();
    counter();
    counter();
    return 0;
}
```

```
}
```

Output:

```
1
1
1
1
```

Static global Variable

```
static int x = 100;
```

Such a variable:

- Exists throughout the program.
- Is visible only within that source file.
- Cannot be accessed from another file.

External Storage class (extern): The external keyword declares a variable that is defined elsewhere, often in another source file. It allows multiple source file to share the same global variable.

```
StorageClasses > C four.C > [?] number
1  #include<stdio.h>
2
3  int number = 100;
```

```
StorageClasses > C five.C > ...
1  #include<stdio.h>
2  #include"four.c"
3
4  extern int number;
5
6  int main(int argc, char const *argv[])
7  {
8      printf("\nnumber = %d\n", number);
9      return 0;
10 }
11
```

Output:

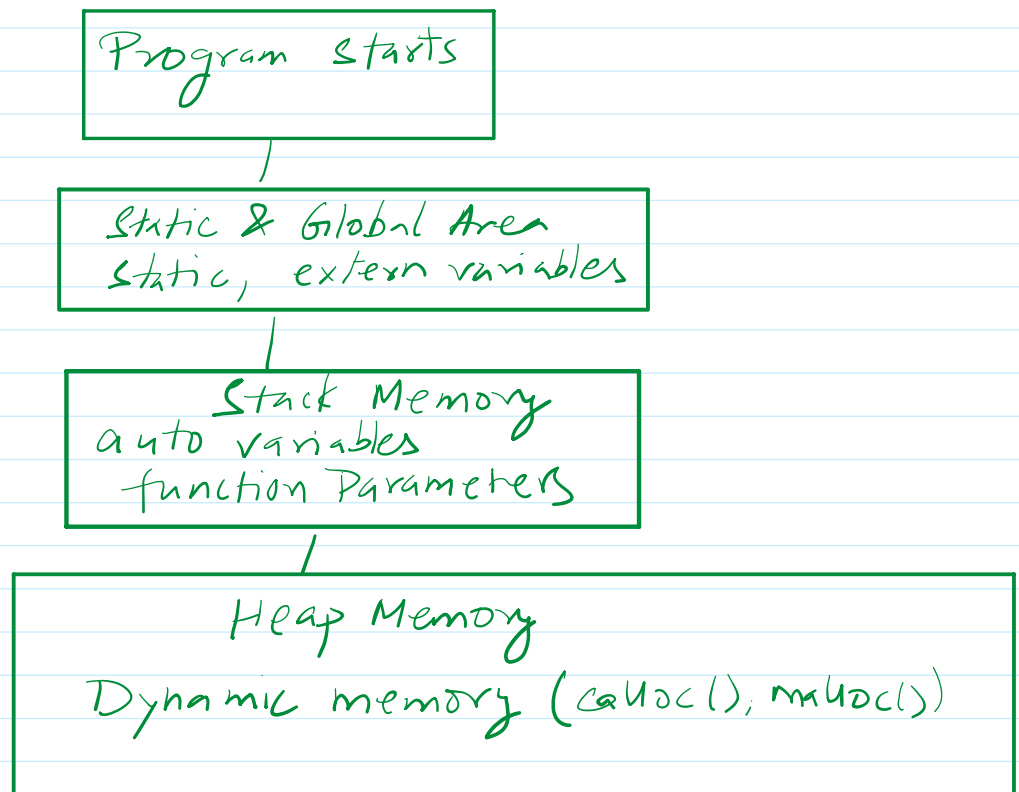
```
number = 100
```

Comparison of storage classes

Feature	auto	register	static	extern
---------	------	----------	--------	--------

Storage Location	RAM	CPU Register(if available)	Data Segment	Data Segment
Scope	Local	Local	Local or Global	Global
Lifetime	Function/Block/execution	Function/block/execution	Entire Program	Entire Program
Default value	Garbage	Garbage	0	0
Can retain value?	No	No	Yes	Yes
Address operator allowed(&)	Yes	No	Yes	Yes

Memory representation



Key differences between static and extern

Static	Extern
File level visibility.	Accessible across files.
Used to hide variables or functions within A file.	Used to share variables or functions across files.
Lifetime: Entire program.	Lifetime: Entire program.

Key difference between auto and static

Auto	Static
Created every function call.	Created only once.
Destroyed after function ends.	Exist throughout the program.

Does not retain value.	It retains value.
Garbage initial value.	Default initialized to zero.